PART B — QUESTIONS WITH SOLUTIONS

Q1. What is a digital image? Differentiate between grayscale and color images.

A digital image is a two-dimensional representation of a visual object consisting of small elements called pixels. Each pixel contains information about the intensity or color at a particular position.

Grayscale Image:

Contains intensity information only. Usually has one channel. In an 8-bit image, pixel values range from 0 to 255. 0 represents black and 255 represents white.

Color Image:

Contains color information. Usually has three channels. RGB images use Red, Green, and Blue channels. Different combinations of the three channels produce different colors.

Q2. Explain the difference between RGB, BGR, HSV, and LAB color spaces.

RGB: RGB stands for Red, Green, and Blue. It represents colors using three channels and is commonly used for displaying images.

BGR: BGR contains Blue, Green, and Red channels. OpenCV normally reads color images in BGR format.

HSV: HSV stands for Hue, Saturation, and Value.

Hue represents the basic color. Saturation represents color intensity or purity. Value represents brightness.

HSV is useful for color detection and object tracking.

LAB: LAB represents:

L — Lightness A — Green to Red component B — Blue to Yellow component

LAB is useful in image analysis and color-related processing.

Q3. What is the purpose of converting an image to grayscale before further processing?

Converting an image to grayscale changes a color image from three channels into a single intensity channel.

The main purposes are:

Reducing the amount of data. Making processing faster. Simplifying calculations. Removing unnecessary color information. Making operations such as edge detection and thresholding easier.

The OpenCV function used is:

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

Q4. Explain the concept of image complement (negative image) and mention its practical applications.

An image complement, also called a negative image, is produced by reversing the intensity values of pixels.

For an 8-bit image:

Negative Pixel = 255 - Original Pixel

For example:

Original Pixel = 0 Negative Pixel = 255

and:

Original Pixel = 255 Negative Pixel = 0

Python implementation:

negative = 255 - image

Applications:

Medical image analysis Image enhancement Document processing Photography Visualization of image details

Q5. Differentiate between image resizing, cropping, and scaling.

Resizing: Resizing changes the dimensions of an image to specified width and height.

Example:

resized = cv2.resize(image, (400,300))

Cropping: Cropping selects a particular portion of an image and removes the remaining portions.

Example:

roi = image[50:250,100:350]

Scaling: Scaling increases or decreases the size of an image using a scale factor.

For example, if an image is scaled by 0.5, its dimensions become half of the original dimensions.

Q6. What is a Region of Interest (ROI), and why is it important in computer vision?

A Region of Interest (ROI) is a selected portion of an image that contains the area to be analyzed.

Example:

roi = image[50:250,100:350]

ROI is important because:

It reduces the amount of data to process. It focuses processing on an important region. It improves processing efficiency. It helps in object detection and recognition. It removes irrelevant background information.

Q7. Why is OpenCV preferred over conventional image processing libraries for computer vision applications?

OpenCV is widely used for computer vision because it provides many efficient functions for image and video processing.

Advantages of OpenCV include:

It is open-source. It supports Python and C++. It provides image reading and writing functions. It supports color-space conversion. It provides geometric transformations. It supports image filtering and enhancement. It provides many computer vision algorithms. It can efficiently process images and videos.

Q8. Explain how image resolution and pixel intensity influence image quality.

Image Resolution refers to the number of pixels present in an image.

For example:

1920 × 1080

represents an image with 1920 pixels in width and 1080 pixels in height.

Higher resolution generally allows an image to represent more spatial details.

Pixel Intensity represents the brightness value of a pixel.

For an 8-bit grayscale image:

0 → Black 255 → White

Values between 0 and 255 represent different levels of gray.

Therefore, image resolution affects spatial detail, while pixel intensity affects brightness and contrast information.

Q9. Mention five real-world applications where basic image preprocessing is an essential step.

Five real-world applications are:

Medical Imaging — preprocessing of X-rays and medical scans. Autonomous Vehicles — processing camera images before detecting roads, vehicles, and pedestrians. Face Recognition — preparing images before face detection and recognition. Industrial Automation — detecting defects in manufactured products. Surveillance Systems — processing video frames before object detection and tracking.

Q10. How do image preprocessing techniques improve the performance of feature extraction and deep learning models?

Image preprocessing prepares raw images before they are provided to computer vision or deep learning models.

It can improve performance by:

Resizing images to a consistent size. Converting images to grayscale when color information is unnecessary. Normalizing pixel values. Reducing unwanted noise. Enhancing contrast. Selecting important regions using ROI. Converting images into a suitable color space.

These techniques help make important image features easier to extract and can help models learn useful patterns more effectively.

PART A — PYTHON IMPLEMENTATION

```
# EXPERIMENT NO. 1
# Fundamental Image Processing Operations using Python & OpenCV

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# 1. UPLOAD IMAGE

print("Please upload your COLOR image...")

uploaded = files.upload()

# Automatically get uploaded file name
filename = next(iter(uploaded))

print("\nUploaded file:", filename)
```

```
# 2. READ IMAGE

image = cv2.imread(filename)

if image is None:
    print("ERROR: Image could not be loaded!")
else:
    print("Image loaded successfully!")

# 3. DISPLAY ORIGINAL IMAGE

rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(rgb_image)
plt.title("Original Image")
plt.axis("off")
plt.show()

# 4. IMAGE PROPERTIES

height, width, channels = image.shape

print("\n========== IMAGE PROPERTIES ==========")
print("Width       :", width)
print("Height      :", height)
print("Channels    :", channels)
print("Data Type   :", image.dtype)
print("Resolution  :", width, "x", height)

# 5. SAVE IMAGE AS JPEG AND PNG

cv2.imwrite("output.jpg", image)
cv2.imwrite("output.png", image)

print("\nJPEG and PNG files saved successfully.")

# 6. GRAYSCALE CONVERSION

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(7, 5))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()

# 7. HSV COLOR SPACE

hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Convert only for displaying with matplotlib
hsv_display = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(hsv_display)
plt.title("HSV Image")
plt.axis("off")
plt.show()

# 8. LAB COLOR SPACE

lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

# Convert only for displaying
lab_display = cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(lab_display)
plt.title("LAB Image")
plt.axis("off")
plt.show()
```

```
# 9. RESIZE IMAGE

resized = cv2.resize(image, (400, 300))

resized_rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(resized_rgb)
plt.title("Resized Image")
plt.axis("off")
plt.show()

# 10. ROTATE IMAGE

height, width = image.shape[:2]

center = (width // 2, height // 2)

rotation_matrix = cv2.getRotationMatrix2D(
    center,
    45,
    1.0
)

rotated = cv2.warpAffine(
    image,
    rotation_matrix,
    (width, height)
)

rotated_rgb = cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(7, 5))
plt.imshow(rotated_rgb)
plt.title("Rotated Image - 45 Degrees")
plt.axis("off")
plt.show()

# 11. HORIZONTAL FLIP

horizontal_flip = cv2.flip(image, 1)

horizontal_flip_rgb = cv2.cvtColor(
    horizontal_flip,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(7, 5))
plt.imshow(horizontal_flip_rgb)
plt.title("Horizontal Flip")
plt.axis("off")
plt.show()

# 12. VERTICAL FLIP

vertical_flip = cv2.flip(image, 0)

vertical_flip_rgb = cv2.cvtColor(
    vertical_flip,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(7, 5))
plt.imshow(vertical_flip_rgb)
plt.title("Vertical Flip")
plt.axis("off")
plt.show()

# 13. IMAGE COMPLEMENT / NEGATIVE

negative = 255 - image
```

```
negative_rgb = cv2.cvtColor(
    negative,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(7, 5))
plt.imshow(negative_rgb)
plt.title("Negative Image")
plt.axis("off")
plt.show()

# 14. REGION OF INTEREST (ROI)
# Automatically select a central ROI

height, width = image.shape[:2]

x1 = width // 4
x2 = 3 * width // 4

y1 = height // 4
y2 = 3 * height // 4

roi = image[y1:y2, x1:x2]

roi_rgb = cv2.cvtColor(
    roi,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(7, 5))
plt.imshow(roi_rgb)
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()

# 15. DISPLAY ALL IMPORTANT RESULTS TOGETHER

plt.figure(figsize=(16, 10))

plt.subplot(2, 4, 1)
plt.imshow(rgb_image)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 4, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 4, 3)
plt.imshow(resized_rgb)
plt.title("Resized")
plt.axis("off")

plt.subplot(2, 4, 4)
plt.imshow(rotated_rgb)
plt.title("Rotated")
plt.axis("off")

plt.subplot(2, 4, 5)
plt.imshow(horizontal_flip_rgb)
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(2, 4, 6)
plt.imshow(vertical_flip_rgb)
plt.title("Vertical Flip")
plt.axis("off")

plt.subplot(2, 4, 7)
plt.imshow(negative_rgb)
plt.title("Negative")
plt.axis("off")
```

```
plt.subplot(2, 4, 8)
plt.imshow(roi_rgb)
plt.title("ROI")
plt.axis("off")

plt.tight_layout()
plt.show()

# 16. FINAL MESSAGE

print("\n================================================")
print("       EXPERIMENT COMPLETED SUCCESSFULLY")
print("================================================")
print("✓ Image Acquisition")
print("✓ Image Properties")
print("✓ JPEG and PNG Storage")
print("✓ Grayscale Conversion")
print("✓ HSV Conversion")
print("✓ LAB Conversion")
print("✓ Image Resizing")
print("✓ Image Rotation")
print("✓ Horizontal Flipping")
print("✓ Vertical Flipping")
print("✓ Image Complement / Negative")
print("✓ Region of Interest (ROI)")
print("================================================")
```

Please upload your COLOR image...

Choose Files No file chosen Cancel upload